

Games

A game is a loop: read what changed, move the world on by a slice of time, draw it. SPRFST gives you the pieces —time in `std.time`, numbers in `std.math`, dice in `std.rand`, pixels in `std.draw`, a window in `app` —and leaves the loop to you.

The shape of the loop

```
use std.time

fn main() {
  var running = true
  var last = time.clock()

  while running {
    let now = time.clock()
    let dt = now - last
    last = now

    update(dt)
    render()

    time.sleep(0.016)    ~~ about sixty frames a second
  }
}
```

`time.clock()` is a monotonic clock in seconds: it never jumps backwards when the system time is corrected, which is what a game loop needs. Passing `dt` into `update` keeps the game running at the same speed whatever the frame rate.

A world that moves

```
use std.io
use std.math

data Vec2 {
  x: Num
  y: Num
}

object Ball {
  pos: Vec2 = Vec2 { x: 20.0, y: 20.0 }
  vel: Vec2 = Vec2 { x: 90.0, y: 60.0 }
  radius: Num = 4.0

  fn step(self, dt: Num, width: Num, height: Num) {
    var x = self.pos.x + self.vel.x * dt
    var y = self.pos.y + self.vel.y * dt
    var dx = self.vel.x
    var dy = self.vel.y
  }
}
```

```

        if x < self.radius { x = self.radius  dx = 0.0 - dx }
        if x > width - self.radius { x = width - self.radius  dx = 0.0 - dx }
        if y < self.radius { y = self.radius  dy = 0.0 - dy }
        if y > height - self.radius { y = height - self.radius  dy = 0.0 - dy }

        self.pos = Vec2 { x: x, y: y }
        self.vel = Vec2 { x: dx, y: dy }
    }
}

fn main() {
    var ball = Ball { }
    let dt = 1.0 / 60.0
    var bounces = 0
    var was = ball.vel.x

    for _frame in 0..240 {
        ball.step(dt, 120.0, 80.0)
        if ball.vel.x != was { bounces = bounces + 1 }
        was = ball.vel.x
    }

    io.say("after 4 seconds the ball is at {math.round(ball.pos.x)}, {math.round(ball.
pos.y)}")
    io.say("it changed direction sideways {bounces} times")
}

```

Two statements on one line are fine when they belong together, as in `x = self.radius dx = 0.0 - dx` — SPRFST ends a statement at the end of a line, not at a semicolon.

Drawing the frame

With the world in variables, drawing is a separate job. In the terminal, a frame is a few lines of text:

```

use std.io

fn frame(bx: Int, by: Int, w: Int, h: Int) -> Text {
    var out = "+" + "-".repeat(w) + "+\n"
    for y in 0..h {
        var row = "|"
        for x in 0..w {
            if x == bx and y == by { row = row + "o" } else { row = row + " " }
        }
        out = out + row + "|\n"
    }
    give out + "+" + "-".repeat(w) + "+"
}

fn main() {
    io.say(frame(3, 2, 20, 6))
    io.say(frame(9, 4, 20, 6))
}

```

On a Mac the same world is drawn into a window with `std.draw` and shown with `app`, or saved as a picture —which is also how you make the title screen:

```
use std.io
use std.draw

fn main() {
  let w = 240
  let h = 160
  let canvas = draw.canvas(w, h)
  draw.clear(canvas, draw.rgb(11, 11, 13))
  draw.rect(canvas, 0, h - 12, w, 12, draw.rgb(28, 26, 24))
  draw.circle(canvas, 70, 90, 10, draw.rgb(255, 161, 54))
  draw.rect(canvas, 150, 60, 10, 46, draw.rgb(120, 200, 255))
  draw.text(canvas, 14, 20, "LEVEL ONE", draw.rgb(242, 242, 240))
  io.say("frame saved: {draw.save_png(canvas, "frame.png")}")
}
```

Dice you can trust

Games need randomness, and tests need the same randomness twice. `rand.seed` fixes the sequence:

```
use std.io
use std.rand

fn roll_three() -> [Int] {
  give [rand.int(1, 7), rand.int(1, 7), rand.int(1, 7)]
}

fn main() {
  rand.seed(2026)
  let first = roll_three()
  rand.seed(2026)
  let again = roll_three()
  io.say("{first} and {again} – the same run twice")

  io.say("a random pick: {rand.choice(["sword", "rope", "lamp"]) ?? "nothing"}")
  io.say("shuffled: {rand.shuffle([1, 2, 3, 4, 5])}")
}
```

State machines beat flags

An `enum` keeps a game honest about what it is doing:

```
use std.io

enum Phase {
  Title
  Playing(Int)
  Paused(Int)
  Over(Int)
}
```

```

fn describe(p: Phase) -> Text {
  give match p {
    when Title -> "press start"
    when Playing(score) -> "playing, score {score}"
    when Paused(score) -> "paused at {score}"
    when Over(score) -> "game over with {score}"
  }
}

fn main() {
  for phase in [Title, Playing(120), Paused(120), Over(340)] {
    io.say(describe(phase))
  }
}

```

The compiler checks that every phase is handled. Add a phase later and it tells you exactly which `match` needs attention.

The example to read next

`examples/20-game.spf` is a complete turn based game: a player object, an event enum, a random encounter table and a win condition.

Exercise

Give the bouncing ball a paddle: a position you move with the keys `a` and `d` read from `io.ask`, one life lost when the ball reaches the bottom, and a score that goes up on every bounce.